

Phân tích bài toán phân hoạch cây

Bei Xiaohui

Nguồn gốc: A brief analysis of tree partitioning problems

IOI China candidate team papers / 2004 / Bei Xiaohui.pdf

Tóm tắt

Bài toán phân hoạch max–min trên cây có thể phát biểu như sau: cho một cây có n đỉnh, mỗi đỉnh có một trọng số không âm, cần chia cây thành k cây con sao cho giá trị nhỏ nhất trong các tổng trọng số của từng cây con là lớn nhất. Một cách làm tự nhiên là chuyển bài toán thành: với một cận dưới cho trước, chia được nhiều nhất bao nhiêu cây con. Kết hợp bài toán mới với tìm kiếm nhị phân sẽ giải được bài toán ban đầu, nhưng độ phức tạp tổng thể phụ thuộc vào miền giá trị của trọng số đỉnh. Bài viết giới thiệu một thuật toán có thời gian không phụ thuộc vào miền trọng số, chứng minh tính đúng đắn, bàn về một vài mở rộng, và cuối cùng so sánh đặc điểm của hai thuật toán.

Từ khóa. Chuyển hóa bài toán; lát cắt; dịch chuyển; phía trên.

1 Đặt vấn đề

Bài toán phân hoạch cây là một lớp bài toán rất rộng trong lý thuyết đồ thị. Nói chung, ta cần chia một cây đã cho thành một số cây con sao cho chúng thỏa một điều kiện nhất định, hoặc làm một hàm mục tiêu nào đó đạt cực trị. Lớp bài toán này đã được mở rộng theo nhiều hướng và có ứng dụng trong nhiều lĩnh vực. Bài viết này tập trung vào một dạng gọi là **phân hoạch max–min**.

1.1 Bài Strawberry

Trong bài *Strawberry* (NOI 2003, ngày 2, bài 2, các bộ test 6–9), ta biết trọng lượng của từng ruộng dâu và quan hệ nối giữa chúng. Ký hiệu $\text{sum}(i)$ là tổng trọng lượng của mọi ruộng dâu trong khối thứ i , với $1 \leq i \leq k$, và đặt

$$x = \min\{\text{sum}(i) \mid 1 \leq i \leq k\}.$$

Cần chia một ruộng dâu thành k khối sao cho:

- trong mỗi khối, các ruộng dâu liên thông trực tiếp hoặc gián tiếp với nhau;
- giá trị x ứng với cách chia là lớn nhất có thể.

Cuối cùng phải xuất cách chia và giá trị đạt được.

Đây là một bài nộp đáp án. Trong các bộ dữ liệu 6 đến 9, đồ thị đã cho là một cây. Nếu bỏ qua chi tiết dữ liệu cụ thể, ta có thể trừu tượng hóa bài toán như sau: cho một cây, mỗi đỉnh có trọng số không

âm. Chia cây thành k cây con. Nếu $\text{sum}(i)$ là tổng trọng số các đỉnh trong cây con thứ i , hãy tối đa hóa

$$x = \min\{\text{sum}(i) \mid 1 \leq i \leq k\}.$$

Nói ngắn gọn, cần chia một cây thành k cây con sao cho cây con có tổng trọng số nhỏ nhất trở nên lớn nhất. Đó là bài toán phân hoạch max–min trên cây.

2 Thuật toán 1: chuyển hóa bài toán

2.1 Chuyển sang bài toán mới

Khi mới gặp bài toán này, ta có thể thấy khó bắt đầu. Những công cụ quen thuộc như quy hoạch động hay tham lam dường như không áp dụng trực tiếp, quan trọng hơn là ta chưa biết giá trị nhỏ nhất cần tối đa hóa là bao nhiêu. Vì vậy, hãy xét trước bài toán sau:

Với một cận dưới đã cố định, nhiều nhất có thể chia cây thành bao nhiêu cây con sao cho tổng trọng số của mỗi cây con đều không nhỏ hơn cận dưới đó?

2.2 Giải bài toán mới

Nếu giải được bài toán này, ta có thể dùng tìm kiếm nhị phân để tìm đáp án của bài toán ban đầu. Trên thực tế, bài toán mới chỉ cần một thuật toán quét dựa trên ý tưởng tham lam.

Thuật toán quét. Quét toàn bộ cây theo thứ tự độ sâu giảm dần. Với mỗi đỉnh đang xét, tính tổng trọng số của cây con đầy đủ nhận đỉnh đó làm gốc. Nếu tổng này không nhỏ hơn cận dưới đã cho, tách cây con đó ra và xóa nó khỏi cây ban đầu. Lặp đến khi tổng trọng số phần còn lại của cây nhỏ hơn cận dưới, rồi nhập phần còn lại vào cây con được tách cuối cùng. Cách chia thu được là một cách chia tối ưu với số lượng cây con lớn nhất.

Chứng minh tính đúng đắn của thuật toán khá đơn giản nên bài gốc lược bỏ. Để thấy thời gian của thuật toán quét là tuyến tính, đạt đến cận dưới lý thuyết cho một lần kiểm tra. Phần còn lại rất trực tiếp: tìm kiếm nhị phân giá trị cận dưới lớn nhất x sao cho số cây con tối đa chia được không nhỏ hơn k . Khi đó x chính là nghiệm của bài toán ban đầu.

2.3 Nhận xét

Ta đã tìm được một hướng giải, nhưng bài toán chưa thật sự được giải một cách hoàn hảo. Khi phân tích độ phức tạp tổng thể, ta thấy nó phụ thuộc vào miền giá trị của trọng số đỉnh. Cụ thể, nếu trọng số mỗi đỉnh không vượt quá c , thì độ phức tạp là

$$O(n \log(nc)).$$

Trong phần lớn trường hợp thuật toán chạy tốt, và với bài Strawberry mọi dữ liệu đều có thể được giải nhanh. Tuy nhiên, khi miền trọng số rất lớn, hoặc trọng số là số thực, cách làm này không còn thỏa đáng. Câu hỏi tự nhiên là: có thể tìm một thuật toán mà độ phức tạp không phụ thuộc vào miền trọng số hay không?

3 Thuật toán 2: thuật toán dịch chuyển

3.1 Ý tưởng mới

Trong một cách phân hoạch, nếu hai đầu mút của một cạnh thuộc hai cây con khác nhau, ta nói trên cạnh đó có một **lát cắt**. Mỗi lát cắt tương ứng với một cây con: đó là toàn bộ các đỉnh ở phía dưới lát cắt ấy sau khi bỏ đi các đỉnh nằm dưới những lát cắt khác. Chỉ cây con chứa gốc là không có lát cắt tương ứng. Vì vậy, bài toán có thể chuyển thành bài toán đặt $k - 1$ lát cắt lên $k - 1$ cạnh sao cho đạt điều kiện tối ưu mong muốn. Trọng tâm được đổi từ chia đỉnh sang phân bố lát cắt.

Ta cũng cần một kỹ thuật gọi là **dịch chuyển**. Một lần dịch chuyển là đưa một lát cắt từ một cạnh sang một cạnh kề với nó, với điều kiện cạnh mới nằm ở tầng dưới cạnh cũ. Nói cách khác, mọi dịch chuyển đều đi từ trên xuống dưới. Nhờ đó, từ một trạng thái phân hoạch ban đầu, qua hữu hạn lần dịch chuyển, ta có thể đạt tới một trạng thái mục tiêu nào đó.

Việc chọn trạng thái ban đầu không khó: lấy tùy ý một đỉnh bậc 1 làm gốc, rồi ban đầu đặt toàn bộ $k - 1$ lát cắt lên cạnh duy nhất kề với gốc. Từ trạng thái này, ta có thể đạt tới mọi trạng thái mong muốn. Điểm mấu chốt là quy tắc chọn mỗi lần dịch chuyển sao cho khi thuật toán dừng, giá trị nhỏ nhất trong các cây con là lớn nhất.

3.2 Mô tả thuật toán

Quy tắc thực ra đơn giản hơn dự đoán, vẫn dựa trên tham lam. Ở mỗi bước, xét tất cả các dịch chuyển khả dĩ và tính tổng trọng số của cây con mới tương ứng với lát cắt sau khi nó đến vị trí mới. Chọn dịch chuyển làm giá trị mới này lớn nhất. So sánh giá trị đó với tổng nhỏ nhất trong các cây con hiện tại; nếu nó không nhỏ hơn giá trị nhỏ nhất hiện tại thì thực hiện dịch chuyển, ngược lại thuật toán kết thúc.

Thuật toán dịch chuyển.

1. Chọn một đỉnh bậc 1 bất kỳ làm gốc, đặt $k - 1$ lát cắt trên cạnh duy nhất nối với gốc.
2. Tính $W_{\min}$, tổng trọng số nhỏ nhất của một cây con trong trạng thái hiện tại.
3. Xét mọi dịch chuyển khả dĩ, tìm dịch chuyển làm tổng trọng số W_{now} của cây con tương ứng với lát cắt sau khi dịch chuyển là lớn nhất.
4. Nếu $W_{\text{now}} \geq W_{\min}$, thực hiện dịch chuyển đó và quay lại bước 2.
5. Thuật toán kết thúc. Khi đó $W_{\min}$ là giá trị max-min cần tìm, và phân hoạch hiện tại là một phân hoạch tối ưu.

Ta vẫn cần chứng minh tính đúng đắn, vì thuật toán dùng nhiều khái niệm mới và không thể chỉ dựa vào trực giác.

3.3 Quan hệ “phía trên”

Khi quan sát các ví dụ nhỏ, ta thấy một hiện tượng thú vị: sau mỗi lần dịch chuyển, trạng thái phân hoạch do thuật toán sinh ra luôn nằm **phía trên** một phân hoạch tối ưu nào đó. Mỗi thao tác kéo phân hoạch hiện tại xuống thấp hơn một chút, cho đến khi nó trùng với một phân hoạch tối ưu. Đây là hướng chứng minh chính: định nghĩa quan hệ “phía trên” giữa các phân hoạch, chứng minh mọi trạng thái của thuật toán đều nằm phía trên một phân hoạch tối ưu, và nếu một trạng thái còn nằm phía trên một phân hoạch tối ưu khác nó thì thuật toán chắc chắn còn tiếp tục.

Định nghĩa trực quan là: phân hoạch A nằm phía trên phân hoạch A' nếu tồn tại một song ánh giữa các lát cắt của A và các lát cắt của A' sao cho mỗi lát cắt của A nằm phía trên lát cắt tương ứng trong A' . Định nghĩa này phản ánh đúng ý nghĩa hình học, nhưng để chứng minh thuận tiện hơn, ta cần một tính chất thực dụng hơn.

Trước hết, gọi T' là một **cây con bộ phận** của T tại đỉnh v nếu T' chứa v , một con nào đó của v , và mọi hậu duệ của người con đó. Cạnh duy nhất nối v với người con này được gọi là cạnh khởi đầu của T' .

Xét hai phân hoạch A và A' trên cây T . Nếu A nằm phía trên A' , thì với mọi cây con bộ phận của T , hễ một lát cắt c của A nằm trong cây con đó, lát cắt c' tương ứng của A' cũng nằm trong cây con đó vì c' ở phía dưới c . Ký hiệu $\#_S(A)$ là số lát cắt của phân hoạch A nằm trong cây con bộ phận S . Ta có tính chất:

$$\#_S(A) \leq \#_S(A')$$

với mọi cây con bộ phận S của T . Tính chất này đóng vai trò quan trọng trong chứng minh dưới đây.

3.4 Chứng minh thuật toán

Để chứng minh tính đúng đắn, ta sẽ chứng minh bốn mệnh đề:

1. Ở trạng thái ban đầu, phân hoạch A nằm phía trên mọi phân hoạch tối ưu Q .
2. Nếu tồn tại một phân hoạch tối ưu Q sao cho phân hoạch hiện tại A nằm phía trên Q và $A \neq Q$, thì thuật toán chắc chắn không dừng.
3. Giả sử A nằm phía trên Q và $A \neq Q$. Sau một bước thuật toán biến A thành A' , ta vẫn tìm được một phân hoạch tối ưu Q' sao cho A' nằm phía trên Q' .
4. Thuật toán dừng sau hữu hạn bước, và khi dừng, phân hoạch thu được là tối ưu.

Mệnh đề 1 hiển nhiên. Mệnh đề 4 cũng dễ thấy: mọi dịch chuyển đều đi từ trên xuống dưới nên các lát cắt không thể dịch chuyển vô hạn. Do đó thuật toán phải dừng sau hữu hạn bước. Nếu trạng thái dừng không tối ưu, theo mệnh đề 3 tồn tại một phân hoạch tối ưu Q sao cho phân hoạch hiện tại A nằm phía trên Q . Vì A không tối ưu nên $A \neq Q$, và theo mệnh đề 2 thuật toán vẫn phải tiếp tục, mâu thuẫn với việc đã dừng.

Trong phần sau, A thường là phân hoạch do thuật toán sinh ra, còn Q là một phân hoạch tối ưu. Ký hiệu $W_{\min}(A)$ là tổng trọng số nhỏ nhất của một cây con trong phân hoạch A . Với mọi phân hoạch A , ta luôn có

$$W_{\min}(A) \leq W_{\min}(Q).$$

Mệnh đề 2. Nếu tồn tại một phân hoạch tối ưu Q sao cho A nằm phía trên Q và $A \neq Q$, thì thuật toán chắc chắn không dừng.

Chứng minh. Vì $A \neq Q$, trong A tồn tại một lát cắt sao cho trên cùng cạnh đó không có lát cắt nào của Q . Gọi c là lát cắt sâu nhất thỏa điều kiện này. Xét cây con bộ phận có cạnh chứa c làm cạnh khởi đầu. Vì A nằm phía trên Q , ta có $\#(A) \leq \#(Q)$ trên cây con bộ phận đó. Lại do trên cạnh của c không có lát cắt của Q , bên trong cây con phải tồn tại một lát cắt s của Q sao cho trên cạnh của s không có lát cắt nào của A .

Lấy lát cắt c' đầu tiên của A gặp được khi đi từ s lên trên. Dịch c' một bước theo hướng đến s . Khi đó tổng trọng số của cây con mới ứng với c' không nhỏ hơn tổng trọng số của cây con ứng với s , và vì Q là tối ưu, ta có

$$W_{\text{new}} \geq W_s \geq W_{\min}(Q) \geq W_{\min}(A).$$

Do đó tồn tại một dịch chuyển hợp lệ không làm giảm giá trị nhỏ nhất hiện tại, nên thuật toán không thể dừng. Hơn nữa, sau mỗi bước dịch chuyển của thuật toán, tổng trọng số của cây con mới ứng với lát cắt được dịch chuyển không nhỏ hơn $W_{\min}(Q)$. Kết luận phụ này sẽ được dùng trong mệnh đề 3.

Mệnh đề 3. Giả sử A nằm phía trên Q và $A \neq Q$. Sau khi thuật toán thực hiện một bước và biến A thành A' , ta vẫn tìm được một phân hoạch tối ưu Q' sao cho A' nằm phía trên Q' .

Chứng minh. Giả sử bước dịch chuyển của thuật toán đưa một lát cắt c từ cạnh e_1 sang cạnh e_2 , và hai cạnh này gặp nhau tại đỉnh v . Ta xây dựng Q' theo hai trường hợp.

Trường hợp 1. Với mọi cây con bộ phận tại đỉnh v , số lát cắt của A và Q trong cây con đó đều bằng nhau. Khi c dịch từ e_1 xuống e_2 , trên cây con bộ phận có cạnh khởi đầu e_2 ta có nhiều hơn một lát cắt của A' so với Q . Vì trước đó trên cây con bộ phận có cạnh khởi đầu e_1 vẫn có $\#(A) \leq \#(Q)$, cạnh e_1 phải chứa một lát cắt s của Q . Dịch s từ e_1 sang e_2 để tạo phân hoạch Q' . Khi đó với mọi cây con bộ phận, A' vẫn nằm phía trên Q' .

Hơn nữa, cây con tương ứng với s trong Q' chứa cây con tương ứng với c sau bước dịch chuyển của thuật toán. Vì kết luận phụ ở mệnh đề 2 cho biết cây con của c có trọng số không nhỏ hơn $W_{\min}(Q)$, cây con của s trong Q' cũng có trọng số không nhỏ hơn $W_{\min}(Q)$. Do đó Q' vẫn là một phân hoạch tối ưu.

Trường hợp 2. Tồn tại một cây con bộ phận S tại đỉnh v sao cho $\#_S(A) < \#_S(Q)$. Nếu ngay trên cây con bộ phận có cạnh khởi đầu e_2 đã có $\#(A) < \#(Q)$, thì sau khi dịch chuyển vẫn có thể lấy $Q' = Q$.

Ngược lại, trên cây con bộ phận có cạnh khởi đầu e_2 ta có $\#(A) = \#(Q)$. Khi đó tồn tại một cạnh e_3 xuất phát từ v sao cho trên cây con bộ phận có cạnh khởi đầu e_3 có $\#(A) < \#(Q)$. Gọi s là lát cắt nông nhất của Q trong cây con đó, rồi dịch s sang e_2 để được phân hoạch Q' . Khi đó A' nằm phía trên Q' .

Tương tự trường hợp 1, cây con mới tương ứng với s có trọng số không nhỏ hơn $W_{\min}(Q)$. Cây con chứa đỉnh v cũng không nhỏ hơn $W_{\min}(Q)$ vì sau thay đổi nó chứa thêm cây con trước đây tương ứng với s . Chỉ hai cây con này thay đổi khi đi từ Q sang Q' , nên Q' vẫn tối ưu và A' nằm phía trên Q' . Mệnh đề được chứng minh.

Đến đây tính đúng đắn của thuật toán đã được chứng minh. Nhìn lại toàn bộ chứng minh, khái niệm “phía trên” giữ vai trò trung tâm. Thực chất đây là một quan hệ thứ tự. Việc đưa quan hệ này vào giúp các trạng thái vốn có vẻ rối rắm trở nên có trật tự, từ đó mở đường cho lời giải; đây là một ý tưởng rất đáng học trong lập trình thi đấu.

3.5 Độ phức tạp

Mặc dù mô tả thuật toán không phức tạp, nếu cài đặt trực tiếp theo đúng quy trình thì hiệu quả thấp. Cần tối ưu hóa các thao tác để từng bước có thể được thực hiện nhanh. Theo bài gốc, độ phức tạp tốt đã biết của thuật toán là

$$O(k^2 \text{rd}(T) + kn),$$

trong đó $\text{rd}(T)$ là bán kính của cây. Chi tiết cài đặt khá phức tạp nên không được trình bày; độc giả quan tâm có thể xem các tài liệu tham khảo liên quan.

3.6 Mở rộng thuật toán

Sau khi có thuật toán, ta có thể hỏi liệu nó áp dụng được cho những hàm trọng số khác không. Trong bài toán gốc, trọng số của một cây con là tổng trọng số các đỉnh của nó. Với các hàm khác, chẳng hạn trọng số lớn nhất của một đỉnh trong cây con, hay đường kính của cây con, thuật toán còn đúng không?

Quan sát chứng minh, chỉ các mệnh đề 2 và 3 dùng đến hàm trọng số của cây con. Đi sâu hơn, chứng minh chỉ cần một tính chất:

$$W(T) \geq W(T')$$

với mọi cây con T' của T , trong đó $W(T)$ là hàm trọng số của cây T . Vì vậy, miễn là hàm trọng số thỏa tính đơn điệu này, chứng minh vẫn đúng và thuật toán vẫn khả thi. Điều này cho thấy thuật toán có khả năng mở rộng mạnh.

Từ mở rộng hàm trọng số, ta có thể nghĩ đến mở rộng bài toán: chẳng hạn chia cây thành k cây con sao cho giá trị lớn nhất của các cây con là nhỏ nhất, hoặc sao cho hiệu giữa giá trị lớn nhất và nhỏ nhất là nhỏ nhất. Có thể dùng nguyên thuật toán trên không? Câu trả lời là không phải lúc nào cũng được, vì thuật toán có đặc thù riêng. Tuy nhiên, tư tưởng của nó vẫn có thể được mượn: trong các bài toán liên quan, ta có thể thay đổi quy tắc dịch chuyển hoặc sửa định nghĩa “phía trên” để thiết kế thuật toán phù hợp. Những hướng này nằm ngoài phạm vi bài viết.

3.7 Dùng thuật toán 1 để tối ưu thuật toán 2

Cuối cùng, bài gốc nhắc đến một cách tối ưu thuật toán 2. Trong thuật toán dịch chuyển, trạng thái ban đầu đặt toàn bộ $k - 1$ lát cắt lên cạnh duy nhất kề với gốc. Ưu điểm của cách này là chắc chắn trạng thái ban đầu nằm phía trên một phân hoạch tối ưu nào đó. Nhược điểm là từ trạng thái đó đến tối ưu thường cần rất nhiều bước dịch chuyển.

Ta có thể chọn trạng thái ban đầu tốt hơn bằng thuật toán quét ở phần đầu. Nếu đặt cạnh dưới là

$$\frac{\text{tổng trọng số mọi đỉnh của cây}}{k - 1},$$

thì số cây con mà thuật toán quét tách ra chắc chắn không vượt quá $k - 1$. Đặt các lát cắt còn lại lên cạnh nối với gốc. Có thể chứng minh trạng thái này vẫn nằm phía trên một phân hoạch tối ưu, nhưng số bước dịch chuyển cần thiết để đạt tối ưu giảm xuống. Nhờ một bước tiền xử lý tuyến tính, hai thuật toán được kết hợp một cách khéo léo.

4 Tổng kết

Bài viết trình bày hai thuật toán cho bài toán phân hoạch max–min trên cây. Chúng nhìn vấn đề từ hai góc độ khác nhau và vì vậy dẫn tới hai lời giải khác nhau. Thuật toán 1 dùng tư tưởng chuyển hóa bài toán: biến bài toán ban đầu thành một bài toán mới dễ giải, rồi kết hợp với tìm kiếm nhị phân. Cách này dễ cài đặt và có hiệu quả thời gian tốt trong nhiều trường hợp.

Thuật toán 2 tập trung vào các lát cắt giữa cây con, nhờ đó thu được một thuật toán có độ phức tạp không phụ thuộc vào miền giá trị trọng số. Quan hệ “phía trên” giữa các phân hoạch, vốn đóng vai trò lớn trong chứng minh, thực chất là một tư tưởng thứ tự hóa trạng thái; tư tưởng này có phạm vi ứng dụng rộng trong lập trình thi đấu.

Các đề thi hiện nay ngày càng linh hoạt, không phải lúc nào cũng có thể giải bằng cách áp dụng máy móc các thuật toán kinh điển. Vì vậy, cần suy nghĩ sâu hơn về bản chất của bài toán; chỉ khi hiểu rõ bản chất đó, ta mới có thể thiết kế được thuật toán phù hợp và hiệu quả.

References

- [1] Ronald I. Becker and Yehoshua Perl. The shifting algorithm technique for the partitioning of trees. *Journal of the ACM*, 28 (1981), 5–15.

- [2] Yehoshua Perl and Stephen R. Schach. Max-min tree partitioning. *Discrete Applied Mathematics*, 62 (1995), 15–34.